

simple_rl v2 — System Architecture

LoRA Training · Adapter Continuity · W&B Observability

OpenClaw-RL Project

2026-03-29

Component Legend

Swatch	Component	Role
	train_async.py	AsyncIO orchestrator, adapter continuity state
	LocalEnvPool / TerminalEnv	Docker container pool
	agent_loop.py	Multi-turn agent, semaphore, retry
	oMLX Policy Server	LLM serving on port 8080
	rollout_buffer.py	GRPO advantage computation
	prm_client.py	Optional per-step scoring + JSONL
	mlx_grpo_bridge.py	Subprocess orchestrator (NEW)
	grpo_worker.py	mlx-tune training subprocess (NEW)
	Weights & Biases	Observability sink
	Dataset	Task definitions

What Is New in v2 (marked <-- NEW in v2 in diagram)

Node	Enhancement
Policy Semaphore	asyncio.Semaphore(1) serialises all oMLX API calls
Exponential Backoff Retry	3x retry on HTTP 500 with 2 s / 4 s / 8 s delays
prm_steps.jsonl	Per-turn PRM votes + representative evaluation text
mlx_grpo_bridge.py	Subprocess bridge; serialises job JSON, reads result JSON
grpo_worker.py	Full mlx-tune training subprocess under dedicated venv
load_adapter(prev_path)	Resumes LoRA weights from previous round's checkpoint
LoRA Verification	Trainable-param count + rank/alpha logged to stdout and W&B
wandb.define_metric	Sets grpo/step as x-axis so loss charts render correctly
_last_adapter_path	Module-level state that chains adapter checkpoints across rounds

Round-to-Round Adapter Continuity Flow

Round 1: from_pretrained(base) -> get_peft_model() [no prev adapter]
 -> train -> save -> logs/grpo_adapters/round_0001/

Round 2: from_pretrained(base) -> get_peft_model()
 -> load_adapter(round_0001/) [fill lora_a, lora_b]
 -> train -> save -> logs/grpo_adapters/round_0002/

Round N: from_pretrained(base) -> get_peft_model()
 -> load_adapter(round_000N-1/) [accumulate all prior updates]
 -> train -> save -> logs/grpo_adapters/round_000N/

Each round's saved adapter is the **cumulative sum** of all gradient updates applied across rounds 1 through N. The base model weights are never modified.